Linux & Bash Scripting

Lecture 1 - SED Introduction and substitution command

Key topics

- Introduction to Sed
- The Substitute Command
- Flags and Delimiters "g", "i", "d" flag

Introduction to Sed

Sed is a powerful stream editor in Unix and Linux systems, perfect for making automated modifications to files. It is widely used in scripts and command-line workflows for its ability to process text line-by-line efficiently.

- Sed operates by reading input line-by-line and applying commands.
- It supports simple text replacements as well as complex transformations.
- Despite its simplicity, sed's documentation often feels overwhelming—don't worry, we'll cover it step by step.

Common Use Cases for Sed

Sed is a versatile tool for many scenarios. Below are some practical applications:

- Search and Replace: Quickly locate and replace text in files, such as modifying credentials or configuration settings.
- Log Sanitization: Remove or alter entries in logs to obfuscate traces of red team activity.
- Mass Text Modifications: Edit multiple files or lines of a file simultaneously.

The Substitute Command

The `s` Command

The `s` command is sed's most used feature, allowing text substitution within a file. Here's a simple syntax breakdown:

```
sed 's/pattern/replacement/flags' < input_file > output_file
```

Basic Example

```
echo "day" | sed 's/day/night/'
# Output: night
```

Tip: Always quote the sed pattern to avoid shell interpretation issues.

Multiple Substitutions

```
# Replace 'hello' with 'hi' and 'world' with 'earth'
echo "hello world" | sed 's/hello/hi/; s/world/earth/'

# Output: hi earth
```

Line Orientation in Sed

Sed processes each line of input independently. By default, it only replaces the first occurrence of a pattern per line.

Example

```
Input File:
one two three, one two three
four three two one
one hundred

Command:
sed 's/one/ONE/' file.txt

Output:
ONE two three, one two three
four three two ONE
ONE hundred
```

Tip: Use the `g` flag to replace all occurrences of a pattern on a line.

Flags and Delimiters (-g)

Global Flag (g)

The `g` flag applies the substitution globally within a line, meaning all occurrences of the search string are replaced:

```
# Input (file.txt):
foo bar foo bar

# Command:
sed 's/bar/baz/g' file.txt

# Output:
foo baz foo baz
```

Flags modify the behavior of the `s` command. For instance, the `g` flag applies the substitution globally on each line

Flags and Delimiters (-i)

In-Place Editing Flag (`-i`)

The `-i` flag enables in-place editing of a file. This means changes are made directly to the file without creating a new file:

```
# Input (file.txt):
apple
foo
banana
foo

# Command:
sed -i 's/foo/bar/g' file.txt

# Output (file.txt after execution):
apple
bar
banana
bar
```

```
# Input:
temp=enabled
temp_directory=/var/temp

# Command:
sed -i 's/temp/permanent/g' settings.conf

# Output:
permanent=enabled
permanent_directory=/var/permanent
```

Note: The `-i` flag modifies the file directly, without the need for output redirection.

Flags and Delimiters (-d)

Delete Flag (`d`)

The `d` flag is used to delete lines that match a given pattern:

```
# Input (file.txt):
apple
orange
pattern
banana
pattern

# Command:
sed '/pattern/d' file.txt

# Output:
apple
orange
banana
```

Note: The `d` flag removes any lines that contain the word "pattern".

Tip: Backup files before using the `-i` flag in case of mistakes.